

Real Estate CRM

Guia técnico para expansão

Como o sistema funciona hoje, onde estão os dados, como adicionar pipelines e quais riscos devem ser resolvidos antes de escalar.

PHP puro

JavaScript vanilla

Neon PostgreSQL

SiteGround

Google Sheets CSV

O que você acabou de assumir

É um CRM Kanban compacto para organizar leads e registrar andamento comercial. O produto começou como um único pipeline de real estate baseado em arquivo e evoluiu para uma plataforma com vários pipelines configurados no banco.

2

arquivos concentram a aplicação:
`index.html` e `api.php`.

2

modelos de armazenamento:
arquivo JSON legado e Neon.

584

registros em cada CSV local de
origem/fase 1.

Resumo executivo

O QUE JÁ FUNCIONA

- Login de um administrador, com sessão persistida no Neon.
- Seleção de pipeline, Kanban, busca, filtros e modal do lead.
- Importação manual do pipeline legado `real_estate`.
- Sincronização de planilhas publicadas para pipelines no Neon.
- Preservação de status e notas internas durante `reimport/sync`.
- Deploy automático por FTP a cada push na `main`.

O QUE LIMITA A EXPANSÃO

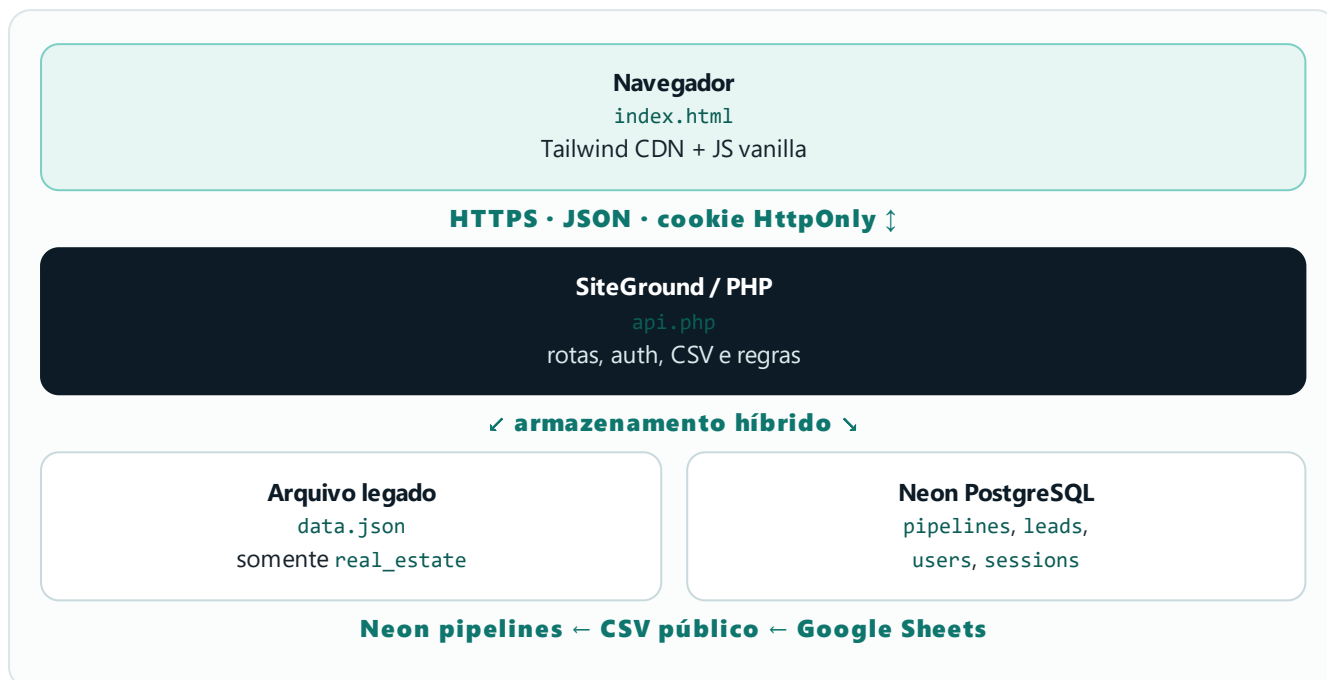
- Não há migrations, schema versionado, testes ou ambiente local reproduzível.
- Frontend e backend são arquivos monolíticos, com regras duplicadas.
- O “auto-sync de 5 min” ocorre novamente a cada carregamento.
- Deploy pode publicar documentação e CSVs com dados pessoais.
- Operação, backup, monitoramento e recuperação não estão documentados.

Fonte de verdade: este guia segue o código atual. `real-estate-crm-spec.md` descreve a primeira versão — sem login, sem Neon e com apenas um pipeline — e hoje deve ser tratado como histórico.

Mapa deste documento

- 02** Arquitetura e responsabilidades
- 03** Fluxo do usuário e tipos de pipeline
- 04** API, autenticação e modelo de dados
- 05** Importação, sincronização e preservação de dados
- 06** Como expandir: configuração versus código
- 07** Deploy, ambiente e operação
- 08** Riscos e correções prioritárias
- 09** Plano recomendado para os primeiros 30 dias
- 10** Checklist de validação e mapa de arquivos

Uma aplicação simples, com dependências operacionais importantes



Responsabilidade de cada camada

Camada	Responsabilidade real	Observação para expansão
<code>index.html</code>	Tela inteira, estado em memória, filtros, renderização, chamadas da API, login e auto-sync.	Tem 1.461 linhas. Qualquer nova tela aumenta acoplamento e risco de regressão.
<code>api.php</code>	Roteamento por <code>?action=</code> , autenticação, leitura/escrita, integração Neon e parsing CSV.	Tem 825 linhas e não usa framework, Composer ou biblioteca de validação.
<code>data.json</code>	Banco de dados em arquivo do pipeline original.	Existe apenas em produção, é ignorado pelo Git e excluído do deploy. Não migrar sem plano e backup.
Neon	Configura pipelines, guarda leads genéricos e mantém login/sessões.	O schema não está no repositório; hoje o banco contém parte essencial da aplicação.
Google Sheets	Origem de dados dos novos pipelines via links CSV publicados.	Os links são públicos para leitura e a sincronização depende da estrutura dos cabeçalhos.
GitHub Actions	Gera <code>config.php</code> com a conexão Neon e publica por FTP.	Push na <code>main</code> é deploy de produção; não há estágio de teste nem homologação.

Princípio central: `real_estate` é um caminho especial e fixo; todos os outros pipelines são dinâmicos. O backend resolve o driver a partir da chave do pipeline.

Dependências externas em tempo de execução

PHP 7.4+ HTTPS `allow_url_fopen` saída HTTPS Tailwind CDN Google Fonts Neon HTTP SQL API

Sem internet, o CSS do Tailwind e a fonte não carregam; sem Neon, login e pipelines dinâmicos ficam indisponíveis. A conexão PostgreSQL não usa PDO: o PHP envia SQL por HTTP para `https://<host-neon>/sql`.

Da abertura do CRM ao trabalho com um lead

Inicialização da aplicação

- 1. Sessão.** O navegador chama `action=session`. Se o admin existe com senha nula, abre o primeiro acesso; se não há sessão válida, abre o login.
- 2. Catálogo.** Após autenticar, chama `action=pipelines`. O backend combina o pipeline fixo `real_estate` com os registros da tabela `pipelines`.
- 3. Leads.** Chama `action=get_leads&pipeline=...`, hidrata o estado do JavaScript e monta as colunas.
- 4. Sync opcional.** Se o pipeline aceita sync e o timestamp parece antigo, o frontend chama `action=sync` antes de exibir o board.
- 5. Trabalho diário.** Busca e filtros rodam no navegador. Ao salvar status/notas, o frontend atualiza a API e depois sua cópia em memória.

Dois tipos de pipeline

Aspecto	Legado: <code>real_estate</code>	Dinâmico: Neon
Origem	Upload manual de CSV pela interface.	Uma ou mais abas do Google Sheets publicadas como CSV.
Persistência	<code>data.json</code> no SiteGround.	Tabelas <code>pipelines</code> e <code>leads</code> .
Campos	Lista fixa de 30 campos + status/notas.	Payload flexível em JSONB + colunas de status/notas.
Atualização	Import/reimport manual.	Sync manual e tentativa de auto-sync.
Exclusão	Desabilitada.	Controlada por <code>supports_delete</code> .
Filtros	Nome, tier e approach.	Busca em nome, email, telefones e <code>Lead_ID</code> .
Modal	Layout específico para qualificação de real estate.	Layout genérico: contato, respostas extras, UTM e notas.

Atenção ao “auto-sync”: para pipelines Neon, `get_leads` devolve `last_synced` vazio. Portanto, a checagem de 5 minutos considera o sync sempre vencido após recarregar a página ou trocar de pipeline. Na prática, isso pode baixar planilhas e executar centenas de upserts em toda abertura.

Comportamentos do Kanban

- Status ausentes da configuração, mas presentes nos leads, são adicionados à lista em memória.
- `board_statuses` limita colunas exibidas — e também as opções do seletor de status. Isso pode impedir mover um lead para um status oculto como “Fechado” ou “Descartado”.
- Cada coluna mostra 15 cards inicialmente; “Carregar mais” adiciona blocos de 15, ainda com todos os dados já carregados no browser.
- Os nomes das colunas podem ser editados no próprio board e são persistidos.

API, autenticação e modelo de dados

Rotas de `api.php`

Ação	Método esperado	Autenticação	Função
<code>session</code>	GET	Pública	Informa login atual ou necessidade de setup.
<code>setup</code>	POST JSON	Pública enquanto senha = null	Define a primeira senha do admin.
<code>login</code>	POST JSON	Pública	Cria token e cookie de sessão.
<code>logout</code>	Frontend usa POST	Pública	Apaga sessão e cookie; backend não restringe o método.
<code>pipelines</code>	GET	Obrigatória	Lista pipeline fixo + configurações Neon.
<code>get_leads</code>	GET	Obrigatória	Retorna todos os leads de um pipeline.
<code>update_lead</code>	POST JSON	Obrigatória	Atualiza status, observação/notas conforme driver.
<code>update_column_labels</code>	POST JSON	Obrigatória	Renomeia visualmente colunas.
<code>import</code>	POST multipart	Obrigatória	Importa CSV no pipeline de arquivo.
<code>sync</code>	POST	Obrigatória	Sincroniza planilhas de pipeline Neon.
<code>delete_lead</code>	POST JSON	Obrigatória	Exclui quando o pipeline permite.

Autenticação atual

IMPLEMENTADO

- Único email fixo: `comercial@oidigitalmedia.com`.
- Hash via `password_hash` e verificação nativa do PHP.
- Cookie `oi_session`: Secure, HttpOnly e SameSite=Lax.
- Validade de 12 h ou 30 dias com "lembrar".
- Bloqueio por 15 min após 5 falhas.

LIMITES

- Somente um administrador; coluna `role` não é aplicada.
- Tokens ficam em texto puro na tabela de sessões.
- Não há MFA, recuperação de senha ou trilha de auditoria.
- Cookie Secure exige HTTPS — login local em HTTP não persiste.

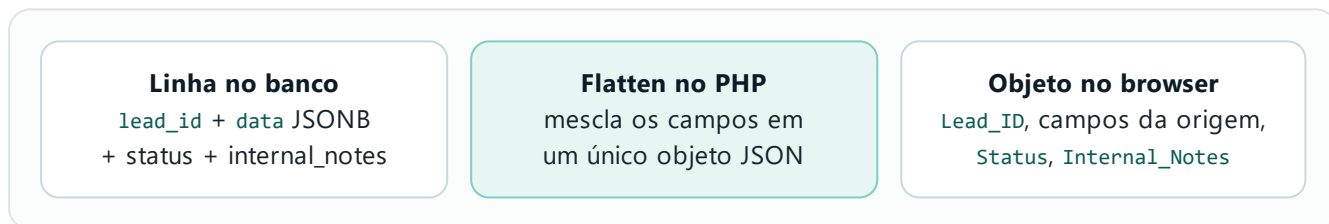
Fronteira de segurança: a chamada a `require_auth()` acontece antes de qualquer leitura ou alteração de pipeline. As quatro ações públicas são sessão, setup, login e logout.

Modelo de dados e configuração

Schema inferido do código

Tabela/arquivo	Campos usados	Chave/contrato
<code>pipelines</code>	<code>key</code> , <code>name</code> , <code>statuses</code> , <code>board_statuses</code> , <code>editable_fields</code> , <code>column_labels</code> , <code>sheet_urls</code> , <code>display</code> , <code>supports_sync</code> , <code>supports_delete</code> , <code>created_at</code>	<code>key</code> identifica o pipeline.
<code>leads</code>	<code>pipeline_key</code> , <code>lead_id</code> , <code>data JSONB</code> , <code>status</code> , <code>internal_notes</code> , <code>created_at</code>	Conflito esperado em (<code>pipeline_key</code> , <code>lead_id</code>).
<code>users</code>	<code>id</code> , <code>email</code> , <code>role</code> , <code>password_hash</code> , <code>failed_attempts</code> , <code>locked_until</code> , <code>last_login_at</code>	Email único esperado.
<code>sessions</code>	<code>token</code> , <code>user_id</code> , <code>expires_at</code> , <code>user_agent</code>	Token usado diretamente no cookie.
<code>data.json</code>	<code>leads[]</code> , <code>last_updated</code> , <code>last_synced</code> , <code>column_labels</code>	<code>Lead_ID</code> é a chave de merge.

Como um registro Neon chega ao frontend



Campos de configuração de pipeline

Campo	Efeito atual	Cuidado
<code>statuses</code>	Lista canônica de etapas.	O backend ainda aceita Status fora dela.
<code>board_statuses</code>	Subset das colunas renderizadas.	Também restringe o dropdown de transição.
<code>column_labels</code>	Nomes visuais editáveis das colunas.	As chaves de status continuam imutáveis.
<code>sheet_urls</code>	Fontes CSV processadas em ordem.	A última aba vence em campos repetidos.
<code>display.field_map</code>	Traduz campos internos para cabeçalhos reais.	<code>title_keys/subtitle_keys</code> não são usados.
<code>supports_sync/delete</code>	Habilita botões e endpoints correspondentes.	São capacidades, não autorização por usuário.

Importante: o DDL exato não está versionado. Antes de alterar tabelas, exporte o schema real do Neon, confirme constraints/índices e transforme-o na migration inicial do projeto.

Como importação e sincronização tratam os dados

Pipeline `real_estate`: upload manual

1. O usuário envia um CSV com cabeçalho e separador vírgula.
2. O backend lê apenas uma lista fixa de 30 campos; campos ausentes viram string vazia.
3. Linhas sem `Lead_ID` são ignoradas.
4. Lead existente é substituído pelos dados novos, mas mantém `Status`, `Observação` e `Internal_Notes`.
5. Novo lead entra com status “Novo”; o conjunto inteiro é gravado em `data.json` com `LOCK_EX`.

Os CSVs locais não são equivalentes ao arquivo final de importação. `real_estate_raw.csv` tem 584 linhas e 51 colunas, porém não possui `Lead_ID` e seria ignorado. `real_estate_fase1.csv` tem 584 linhas e 12 colunas (99 A, 194 B, 291 C), então importa apenas a fase inicial e deixa campos posteriores vazios.

Pipelines Neon: sync de planilhas

- Baixa cada URL de `sheet_urls` sequencialmente, com timeout de 20 s.
- Converte cada linha em objeto usando os cabeçalhos originais.
- Agrupa por `Lead_ID`; abas posteriores sobrescrevem campos de mesmo nome.
- Gera um UPSERT por lead e envia todos como uma transação pela Neon HTTP SQL API.
- No conflito, atualiza somente `data.status` e `internal_notes` permanecem.

Semântica que você precisa preservar — ou mudar conscientemente

Caso	Comportamento atual	Impacto
Lead removido da planilha	Permanece no Neon.	O sync é incremental, não espelha exclusões.
Lead excluído no CRM	Volta como “Novo” no próximo sync se continuar na planilha.	Exclusão não é definitiva sem alterar a origem ou criar tombstone.
Lead só em aba secundária	Também é criado.	Apesar do comentário “LEFT JOIN”, o código faz união por <code>Lead_ID</code> .
<code>Lead_ID</code> duplicado	Última ocorrência/aba vence campo a campo.	Ordem de abas e linhas interfere no resultado.
Status via API	Não é validado contra a lista permitida.	Clientes externos podem criar colunas/status inesperados.
Concorrência no JSON	O lock cobre a escrita, não todo o read-modify-write.	Duas alterações simultâneas podem perder a atualização anterior.

Boa propriedade: status comercial e notas internas ficam separados do JSONB importado no Neon; por isso o sync atualiza informações da planilha sem apagar o trabalho do time.

Quando basta configurar e quando é preciso programar

Cenário A — novo pipeline parecido com os atuais

É possível adicionar sem deploy, desde que exista `Lead_ID` estável e o modal genérico seja suficiente.

1. Publicar cada aba do Google Sheets em formato CSV. Evitar dados que não podem ficar acessíveis por link.
2. Padronizar `Lead_ID` em todas as abas; definir qual aba deve ter precedência nos campos repetidos.
3. Criar o registro em `pipelines` no Neon.
4. Configurar `statuses`, `board_statuses`, URLs, flags e `display.field_map`.
5. Entrar no CRM, sincronizar e validar contagem, campos, status e notas.

```
INSERT INTO pipelines
(key, name, statuses, board_statuses, sheet_urls,
 supports_sync, supports_delete, display)
VALUES
('novo_pipeline', 'Novo Pipeline',
['"Novo","Contatado","Qualificado","Fechado"]',
['"Novo","Contatado","Qualificado","Fechado"]',
['"https://...&output=csv"]', true, true,
{"field_map":{"Nome":"Nome completo",
               "Number":"WhatsApp",
               "DateTime":"Data de entrada"}}::jsonb);
```

Divergência da documentação: `title_keys` e `subtitle_keys` são citados em `PIPELINES.md`, mas o frontend atual não os consulta. O único recurso de `display` efetivamente usado é `field_map`.

Cenário B — novo tipo de card, campos editáveis ou regra de negócio

Exige alteração de código. Hoje o frontend escolhe apenas entre “real estate” e “qualquer driver Neon”; as funções ainda se chamam `renderLaunchLeadCard` e `renderLaunchModal`, revelando a origem específica dessa tela genérica.

Necessidade	Onde mexer hoje	Direção recomendada
Novo visual de card/modal	<code>index.html</code>	Criar renderers por <code>display.type</code> ou schema declarativo.
Novo campo editável	Frontend + <code>neon_update_lead</code> + schema	Validar whitelist no backend e versionar migration.
Nova fonte de dados	<code>api.php</code>	Separar adapters de origem: CSV, webhook, API etc.
Vários usuários/equipes	Auth, schema e todas as queries	Adicionar tenant, papéis e autorização por pipeline.
Mais volume	API e frontend	Paginação/busca no servidor e sync assíncrono em lotes.

Contrato mínimo para qualquer novo pipeline

key imutável Lead_ID estável status “Novo” cabeçalhos estáveis URLs CSV válidas política de exclusão responsável pela origem

Ambiente, deploy e recuperação

Deploy atual

Push em main
GitHub

GitHub Actions
checkout + config.php

FTP
/public_html/

Secrets: FTP_HOST • FTP_USER • FTP_PASS • NEON_DATABASE_URL

- O workflow cria `config.php` com uma constante contendo a connection string do Neon.
- `data.json` é excluído do deploy, evitando sobrescrever o banco legado de produção.
- Não há build, lint, teste, backup, smoke test ou aprovação manual antes da publicação.

Risco crítico de exposição: a regra de FTP exclui apenas Git e `data.json`. Assim, `CLAUDE.md`, specs, este tipo de documento e principalmente `dados/*.csv` podem ser enviados para `public_html`. Os CSVs contêm nomes, emails e telefones. Corrija a lista de exclusão ou, de preferência, publique somente uma pasta de artefatos explicitamente montada.

O que falta para um ambiente local reproduzível

1. Um `README.md` com requisitos, comandos e troubleshooting.
2. Schema/migrations do Neon e dados seed para admin/pipeline de exemplo.
3. `config.example.php` sem segredo e carregamento por variável de ambiente.
4. HTTPS local ou cookie Secure configurável apenas em desenvolvimento.
5. PHP disponível com acesso HTTPS via streams (`allow_url_fopen` + OpenSSL).
6. Planilha/CSV de teste sem dados pessoais.

Runbook mínimo de incidente

Sintoma	Primeiro diagnóstico	Recuperação
Login não abre	Verificar Neon, <code>NEON_DATABASE_URL</code> , HTTPS e tabelas <code>users/sessions</code> .	Restaurar conexão/schema; não zerar senha sem autorização.
Só aparece Real Estate	<code>pipelines</code> falhou ou não retornou registros.	Testar Neon e consulta; o backend oculta falha nessa listagem.
Sync falha	Abrir a URL CSV, conferir <code>Lead_ID</code> , tempo de resposta e headers.	Manter dados já salvos; corrigir origem e sincronizar novamente.
JSON corrompido	API retorna "data.json inválido".	Restaurar backup íntegro antes de qualquer reimport.
Deploy quebrou a tela	Inspecionar último workflow e console/rede do browser.	Reverter por commit e disparar novo deploy; preservar <code>data.json</code> .

Backup obrigatório antes de expansão: snapshot do Neon + cópia versionada fora de `public_html` do `data.json`. Hoje não existe procedimento de backup registrado no repositório.

O que corrigir antes de aumentar uso, dados ou equipe

Prioridade	Risco	Consequência	Ação recomendada
P0	Deploy publica arquivos fora da aplicação.	Exposição de PII e documentos internos.	Deploy por allowlist; remover dados/ do webroot e do artefato.
P0	Sem migrations/schema no Git.	Não é possível recriar banco, ambiente ou recuperação confiável.	Exportar DDL, criar migration inicial e documentar seeds.
P0	Backups não documentados/testados.	Perda definitiva de status, notas e configuração.	Automatizar backup e executar teste de restauração.
P1	<code>last_synced</code> não persiste para Neon.	Sync pesado em cada carregamento.	Persistir timestamp no pipeline ou mover sync para job/webhook.
P1	Status não validado no backend.	Dados inválidos e colunas inesperadas.	Validar contra <code>statuses</code> e campos editáveis.
P1	<code>board_statuses</code> restringe também transições.	Status finais configurados podem ficar inalcançáveis.	Separar status exibidos das opções permitidas.
P1	Sem testes e sem staging.	Todo push em main pode quebrar produção.	Adicionar CI, testes de contrato e ambiente de homologação.
P1	Erros internos retornados ao cliente.	Vazamento de detalhes de banco/rede.	Logar internamente e devolver mensagem/código genérico.
P1	SQL por requisição para todos os leads.	Timeout e payload grande ao crescer.	Batch, staging table/COPY, job assíncrono e idempotência.
P2	Arquivos monolíticos e regras duplicadas.	Mudanças lentas e regressões difíceis de isolar.	Separar módulos sem trocar toda a stack de uma vez.
P2	Todos os leads carregados no browser.	Queda de desempenho com milhares de registros.	Paginação, busca e contagens no servidor.
P2	Único admin, sem auditoria.	Sem responsabilização ou acesso por equipe.	RBAC, histórico de mudanças e eventos de auditoria.

Segurança: decisões para a próxima versão

CURTO PRAZO

- Não publicar dados/documentos.
- Validar origem, método, status e payload.
- Hash de tokens de sessão no banco.
- Rate limit por IP + conta, sem facilitar DoS.
- Sanitizar mensagens de erro externas.

QUANDO HOUVER MAIS USUÁRIOS

- Autorização real por função e pipeline.
- MFA e recuperação de conta.
- Auditoria de status, notas, exclusões e sync.
- Política de retenção e consentimento para PII.
- Revogação e limpeza de sessões expiradas.

Primeiros 30 dias como responsável



Dias 1–3 • Tornar recuperável.

- Identificar responsáveis por SiteGround, Neon, GitHub e planilhas.
- Fazer snapshot do Neon e backup do `data.json`; testar leitura dos backups.
- Exportar o schema e documentar variáveis/secrets sem copiar valores.
- Corrigir imediatamente o escopo do deploy e retirar CSVs do webroot.



Dias 4–10 • Tornar reproduzível.

- Criar README, migrations, seed seguro e ambiente local HTTPS.
- Adicionar lint de PHP/JS e testes dos contratos principais.
- Criar staging com planilha sintética; bloquear deploy direto sem CI.



Dias 11–20 • Tornar previsível.

- Persistir `last_synced` e instrumentar duração/contagens/erros.
- Validar status e corrigir relação entre `statuses` e `board_statuses`.
- Definir semântica de exclusão e de leads ausentes da planilha.
- Alinhar `PIPELINES.md` ao que `display` realmente suporta.



Dias 21–30 • Preparar expansão.

- Extrair configuração, API, drivers e componentes do frontend em módulos.
- Criar template de onboarding de pipeline e checklist de aceite.
- Medir volume esperado e decidir quando introduzir paginação/job assíncrono.
- Planejar multiusuário, auditoria e governança de PII conforme necessidade real.

Ordem segura de refatoração

1 • CONGELAR CONTRATOS

Testes de caracterização para os endpoints e merges atuais.

2 • EXTRAIR MÓDULOS

Auth, drivers, sync e renderers, preservando respostas.

3 • EVOLUIR PRODUTO

Novos campos, usuários, fontes e automações sobre base testada.

Decisões que precisam de dono de negócio

- A planilha ou o CRM é a fonte final quando um lead é excluído?
- Status finais devem desaparecer do board ou continuar navegáveis?
- Quem pode ver/exportar dados pessoais e por quanto tempo?
- Cada pipeline pertence a um cliente/equipe? Há isolamento necessário?
- O sync deve ser por abertura, agenda, botão, webhook ou combinação?
- Qual é a meta de volume, usuários simultâneos e disponibilidade?

Recomendação de arquitetura: mantenha PHP + PostgreSQL no primeiro ciclo e reduza risco com modularização, migrations e testes. Uma reescrita de stack só deve acontecer depois de medir limites concretos.

Checklist de aceite para cada mudança

Funcional

- ❑ Setup inicial funciona somente quando permitido.
- ❑ Login, lembrar, expiração e logout funcionam.
- ❑ Todos os pipelines esperados aparecem.
- ❑ Contagem antes/depois do sync é coerente.
- ❑ Status e notas sobrevivem a reimport/sync.
- ❑ Lead novo, alterado, ausente e duplicado são testados.
- ❑ Busca, filtros, colunas e “carregar mais” funcionam.
- ❑ Exclusão segue a política definida.

Operacional e segurança

- ❑ Backup e restauração foram testados.
- ❑ Artefato de deploy contém somente arquivos públicos.
- ❑ Nenhum secret ou PII aparece em logs/artefatos.
- ❑ Falhas de Neon/Sheets geram erro observável e seguro.
- ❑ Rollback foi ensaiado em staging.
- ❑ Schema e documentação acompanham a mudança.
- ❑ Smoke test roda após deploy.
- ❑ Métricas de sync e erros são acompanhadas.

Mapa dos arquivos do repositório

Arquivo	Uso	Confiabilidade como documentação
index.html	Frontend completo e contrato visual.	Fonte de verdade.
api.php	Backend, auth, drivers, sync e rotas.	Fonte de verdade.
.github/workflows/deploy.yml	Publicação em produção.	Fonte de verdade do deploy.
PIPELINES.md	Operação de pipelines no Neon.	Útil, mas <code>title_keys/subtitle_keys</code> não estão implementados.
CLAUDE.md	Contexto e regra de preservação do legado.	Útil, parcialmente defasado sobre sync.
real-estate-crm-spec.md	Especificação da versão inicial.	Histórico; contradiz login e multi-pipeline atuais.
dados/*.csv	Base bruta e fase 1 do trabalho de qualificação.	Dados de origem; não publicar.
image.png	Captura de uma interface anterior.	Referência visual histórica, não estado atual.

Limites desta análise

Revisão estática do checkout no commit `2450017`. Não houve acesso ao Neon, SiteGround, secrets, planilhas publicadas ou `data.json` de produção. O ambiente local também não traz PHP nem suíte de testes; por isso o comportamento foi confirmado por leitura de código e histórico Git, não por teste ponta a ponta.

Regra de ouro: trate o `data.json` de produção e o schema real do Neon como ativos críticos. Faça backup, versionamento estrutural e teste de restauração antes de expandir funcionalidade ou equipe.